CASSANDRA Hardware Demonstrator: Predicting and Containing a Cyber-to-Physical Attack

Youssef Mhamdi, Mayssam Hergli, Rami Troudi, Eya Jendoubi, Youssef Atig
National Institute of Applied Sciences and Technology (INSAT), Tunis, Tunisia
Technical note accompanying the IASTAM 6.0 Track 5, Problem 10 submission

***Abstract*—This note documents the hardware demonstrator used to show CASSANDRA's forecast, contain, reroute and heal loop on Scenario A (cyber-to-physical cascade). A microcontroller plays satellite B and drives status LEDs for a three-satellite chain. A malicious telecommand latches B's heater; CASSANDRA forecasts that B will breach its thermal limit, flags it, quarantines it and reroutes traffic before the limit is reached, commands safe mode, and restores the node once it is stable. We describe the four software processes and the data bus that connects them, the node-state code shown on the LEDs (G in service, R flagged, O quarantined), the wiring, and the attack timeline step by step.**

I. Purpose and Scope

The demonstrator turns one scenario of the CASSANDRA design into something an audience can watch: an attack that would end in a satellite failure is predicted and contained before the failure happens, while the neighbouring satellites stay in service.

What is real. The attack is a telecommand sent to physical hardware. Telemetry travels over a real serial link and a real publish/subscribe bus. Every decision is taken by running software, and every action (safe-mode command, LED state) is executed on the board.

What is emulated. Satellite B's bus temperature is modelled on the microcontroller rather than read from a physical sensor, and satellites A and C are represented by their status LEDs. The forecaster is a transparent trajectory extrapolator standing in for the temporal graph network of the full design.

II. System Overview

Fig. 1 shows the architecture. The board is connected to a Windows laptop by USB. A PowerShell bridge owns the serial port and relays data to a Redis bus running in Docker inside WSL (Ubuntu). CASSANDRA's monitor and predictor, and the attacker, are Python processes that only talk to the bus. The bus plays the role of the flight software bus: no process talks to another directly.

III. The Four Terminals

Table I summarises the four processes that run during the demonstration.

TABLE I
Processes Running During the Demonstration

Terminal	Role
PowerShell bridge	Reads B's telemetry from the serial port and publishes it on the bus; watches the node states on the bus and drives the LEDs; forwards heater commands (attack or safe mode) to the board. Prints one status line per second.
Ubuntu T1 monitor	Compares what B *reports* (`telemetry:*`) with an independent reference (`truth:*`). Raises an alarm on a mismatch (masking, Scenario B). In Scenario A both agree, so it stays silent: the alert that follows comes from prediction, not from a masking check.
Ubuntu T2 predictor	Forecasts B's time to its critical limit every second, flags B, quarantines it, reroutes traffic, commands safe mode, observes and heals.
Ubuntu T3 attacker	Sends the malicious `HEATER_ON` telecommand and prints B's temperature every second.

A. PowerShell Bridge

The bridge is the only process with access to the serial port. Every 100 ms it (i) reads telemetry lines from the board and writes B's temperature to the bus, (ii) reads the attack/safe-mode key and sends `H` (heater on) or `C` (heater off) to the board when it changes, and (iii) reads the three node states and sends LED commands when they change. Its console line shows the whole constellation at a glance, for example:

```
chip=41.37C sat_temp=41.37C SAT_A:G SAT_B:R SAT_C:G
```

B. Monitor (T1)

The monitor checks cross-domain consistency twice per second. A mismatch larger than a tolerance (3 °C for temperature) on two consecutive checks triggers contain → observe → heal. It is part of the running system in the demonstration to show that it does *not* fire when the telemetry is honest.

C. Predictor (T2)

Once per second the predictor estimates B's temperature rate over the last eight samples, computes the time to the 60 °C limit $t_c = (60 - T)/\dot{T}$, and maps it to a failure-horizon score $\hat{P} = \mathrm{clip}(1 - t_c/30\,\mathrm{s}, 0, 1)$. It prints a `FORECAST` line whenever $\hat{P} > 0$, flags B at $\hat{P} \geq 0.30$, and quarantines it at $\hat{P} \geq 0.85$. While B is quarantined it prints `OBSERVE` lines and heals B once its temperature has stayed below 30 °C for 3 s.

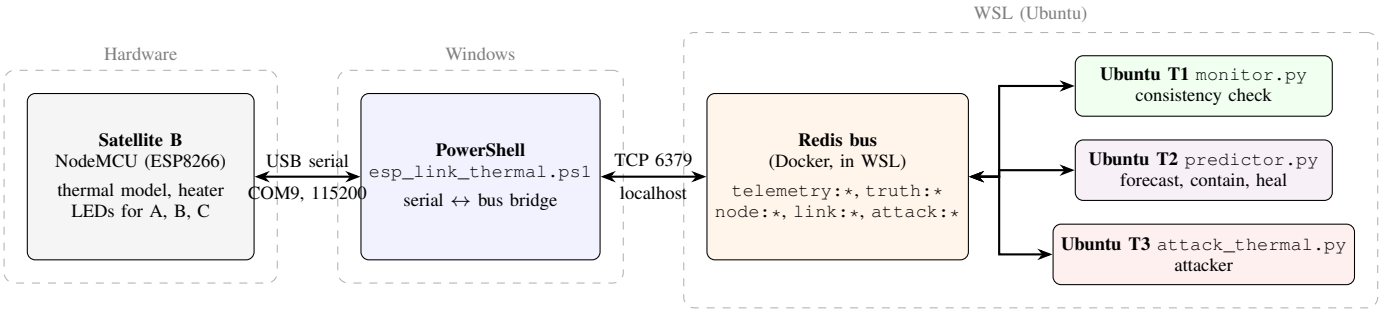


Fig. 1. Demonstrator architecture. Only the bridge touches the serial port; all other processes exchange data exclusively through the Redis bus.

TABLE II
BUS KEYS USED BY THE DEMONSTRATOR

Key	Meaning (writer)
`telemetry:temp`	Temperature B reports (bridge)
`truth:temp`	Independent reference (bridge)
`hw:heater`	Heater state read back from B (bridge)
`attack:thermal`	1 = heater latched, 0 = released (attacker; predictor on safe mode)
`node:sat_a/b/c`	`green`, `red` or `off` (predictor, monitor)
`link:A-B, B-C, A-C`	`up`, `down` or `idle` (predictor)

TABLE III
NODE-STATE CODE

Code	Bus value	LEDs	Meaning
G	`green`	green on	In service, nominal
R	`red`	red on	Flagged: abnormal trend forecast, node still in service
O	`off`	both off	Quarantined: isolated from the chain, links cut

D. Attacker (T3)

The attacker writes the heater command to the bus, which the bridge forwards to the board as `H`. This models a malicious telecommand accepted through abused ground trust (SPARTA IA-0009). `attack_thermal.py stop` releases it.

E. Bus Keys and Serial Protocol

Table II lists the bus keys. On the serial link the board sends `TELEM sim=<°C> heater=<0|1> heap=<bytes>` at 2 Hz and accepts `<idx><state>` LED commands (`idx` 1–3 for A, B, C; `state` `G`, `R` or `O`) and the heater commands `H` and `C`.

IV. NODE-STATE CODE

Each satellite has a green and a red LED. The node state on the bus is translated by the bridge into a one-letter code, shown both on the LEDs and in the bridge's console line (Table III). The board's built-in blue LED is lit while B's heater is latched.

V. WIRING

Fig. 2 shows the circuit. For each satellite, the green and red LEDs are driven by two GPIO pins; their cathodes are joined and share one 150 Ω resistor to ground, which is sufficient because only one of the two is ever lit. With a 3.3 V output and a forward voltage of about 2 V, the current is about 9 mA, within the ESP8266 pin rating. Pins D3, D4 and D8 are left unconnected because they set the boot mode of the chip: loading them with an LED prevents the board from starting.

TABLE IV
BREADBOARD LAYOUT

Sat	Green LED (long, short)	Red LED (short, long)	Resistor	Jumpers
A	c2, c3	d3, d4	a3 → –	D1 → e2, D2 → e4
B	c10, c11	d11, d12	a11 → –	D5 → e10, D6 → e12
C	c18, c19	d19, d20	a19 → –	D7 → e18, D0 → e20

Ground: board G → – column. D3, D4, D8, 3V3, RX, TX: unconnected.

Table IV gives the breadboard layout used (half-size board, left half, columns a–e connected within each row; the "–" column is the ground rail).

VI. SCENARIO A, STEP BY STEP

Satellite B rests at a 25 °C ambient. When the heater is latched its temperature rises at 1.5 °C s^{-1}; the critical limit is 60 °C. Fig. 3 plots the resulting temperature and B's state, and Table V lists what each terminal shows at each stage. Times are measured from the attack.

The key point for the audience is the gap between stages 4 and 5 and the critical limit: B is flagged while it is still more than 20 °C below the limit, and it is isolated ≈4 s before the limit would have been crossed. The limit is never reached, so a conventional threshold alarm on temperature would not have fired at all. Satellites A and C stay in service throughout. Fig. 4 shows an excerpt of the predictor's output for one run.

VII. RUNNING THE DEMONSTRATION

Start-up. (1) In Ubuntu: `cd ~/cassandra-demo && bash reset.sh` to start a clean bus. (2) T1: `pyc cassandra/monitor.py`. (3) T2: `pyc cassandra/predictor.py`. (4) PowerShell, in the folder holding the script: `powershell -ExecutionPolicy Bypass -File .\esp_link_thermal.ps1 -Port COM9`; all three satellites light green. (5) T3: `pyc attacks/attack_thermal.py`.

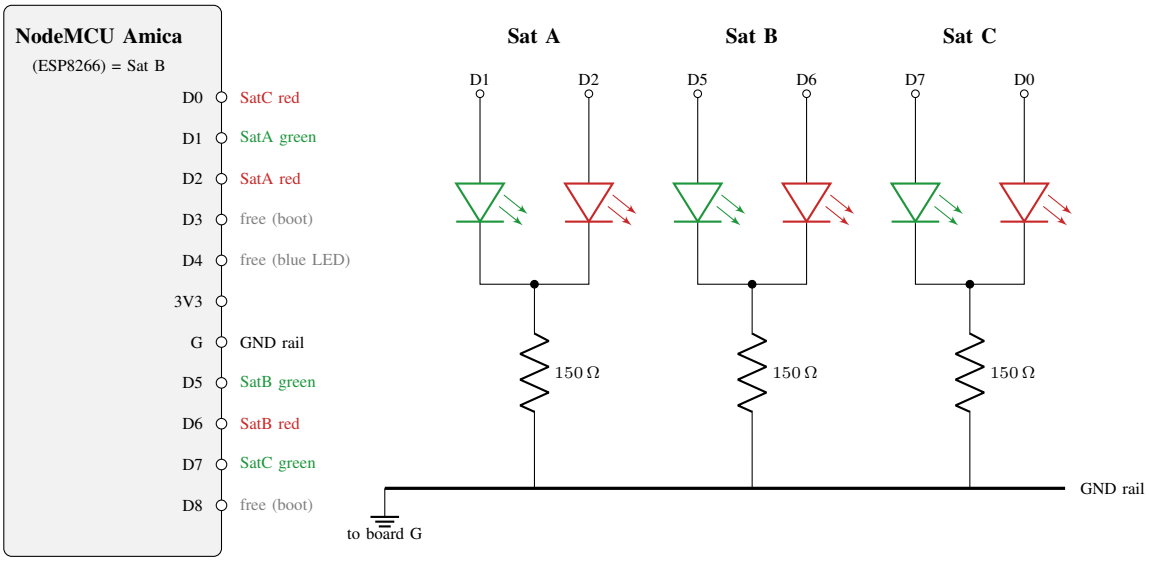


Fig. 2. Wiring. Each satellite's green and red LEDs are driven by two GPIO pins (net labels at the top); the joined cathodes share one 150 Ω resistor to the ground rail, which is connected to a G pin of the board. The long leg of each LED (anode) goes to the pin.

TABLE V
SCENARIO A TIMELINE: TERMINALS AND LEDS

Stage	Time	T_{bus}	A	B	C	What the terminals show
1. Nominal	before 0	25 °C	G	G	G	Bridge prints ≈25 °C each second; monitor and predictor silent.
2. Attack	0 s	25 °C	G	G	G	T3: `malicious HEATER_ON telecommand -> SatB`. Board's blue LED lights; temperature starts rising.
3. Forecast	≈5 s	≈32 °C	G	G	G	T2: `FORECAST` lines with t_c and $\hat{P}$ rising every second.
4. Flag	≈6 s	≈34 °C	G	R	G	T2: `WARNING -> abnormal thermal trend on SatB`. B's red LED on, although the limit is still 26 °C away.
5. Contain	≈19 s	≈54 °C	G	O	G	T2: `ALARM`, `QUARANTINE` ($\hat{P} \geq 0.85$), `REROUTE` (A–B, B–C cut; A–C open), `SAFE-MODE`. B's LEDs off; blue LED off (heater released).
6. Observe	19–27 s	falling	G	O	G	T2: `OBSERVE` lines; B cools while isolated. T3 shows `heater=0`.
7. Heal	≈27 s	<30 °C	G	G	G	T2: `VERIFY`, `HEAL` (links restored, A–C bypass on standby). B's green LED back on.

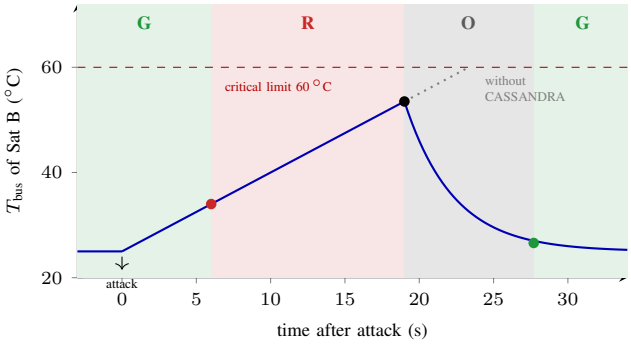


Fig. 3. Temperature of satellite B (computed from the firmware model and controller settings) and its node state. Dots: flagged, quarantined, healed. Without the response the temperature would reach the limit at about 23 s.

Reset between runs. Stop the attacker (Ctrl+C) and run `pyc attacks/attack_thermal.py stop`. The predictor re-arms itself after healing and does not need restarting.

Checks. If the bridge prints nothing, close any program holding the COM port (Arduino IDE serial monitor). If an LED stays dark it is reversed; if the red LED lights instead of the green one, swap that satellite's two jumpers.

```
FORECAST -> temp=34.1C | t_crit=19.9s | P=0.34
WARNING -> abnormal thermal trend on SatB: flagged red
FORECAST -> temp=53.6C | t_crit= 4.3s | P=0.86
ALARM -> cascade projected to breach 60C
QUARANTINE -> SatB P(fail<30s)=0.86: isolating node
REROUTE -> cut A-B & B-C; opened feasible A-C bypass
SAFE-MODE -> heater telecommand revoked; SatB cooling
OBSERVE -> SatB isolated, temp=29.7C (1/3s below 30C)
VERIFY -> SatB at 27.5C, below 30C for 3s: nominal
HEAL -> SatB restored; A-B & B-C links back up
```

Fig. 4. Predictor (T2) output during one run (abridged; lines shortened).

VIII. LIMITATIONS OF THE DEMONSTRATOR

The thermal channel is emulated, so the demonstrator shows the decision loop, not sensor physics. The forecaster uses one signal and a fixed horizon, and its score is a heuristic rather than a calibrated probability. Satellites A and C are represented by LEDs rather than independent nodes, so rerouting is shown as a change of link state on the bus. The monitoring services run on a ground laptop rather than on the onboard computer.

These are the gaps the full design (graph forecaster, calibrated $P(F < t + \Delta)$, onboard execution) is meant to close.